# *Complexity Analysis – USEI07*

This section presents the complexity analysis of the KDnode and KDtree classes implemented for USEI07.
The analysis follows the ESINF methodology: first defining the recurrence T(n) when applicable and then presenting the asymptotic complexity using O(·) notation.

All code involved is deterministic, meaning that the running time depends only on the number of stations *n* and not on the order or distribution of the input.
Therefore, the expected case coincides with the worst case, which is the relevant case for this analysis.

Let:

- **n** = number of railway stations
- **m** = number of KD-tree nodes (each node groups all stations sharing the same coordinates)
- **s** = average number of stations stored in each node (n / m)

## 1- KDnode classe:

Represents a node in the KD-tree that stores one coordinate pair and all stations located exactly at that point.

1.1 – Constructor:

```
public KDnode(double latitude, double longitude,
              List<RailwayStation> sameCoordStations, int
axis) {
    this.coords = new Point2D.Double(longitude, latitude);
    this.axis = axis;
    this.stations = new ArrayList<>(sameCoordStations);
    this.stations.sort(Comparator.comparing(
            RailwayStation::getName,
            String.CASE_INSENSITIVE_ORDER
    ));
}
```

- Point2D.Double creation: O(1).
- ArrayList copy: O(s).
- Sorting stations by name: O(s log s)

Explanation for the sorting complexity:

Sorting **s** stations by name uses Java's TimSort.
Since comparison-based sorting has a lower bound of $\Omega(s\log\ s)$, the expected time is:

$$O(s\log\ s)$$

- Total: O(s log s).

1.2-Getters and Setters:

```
public double getLatitude() { return coords.y; }
public double getLongitude() { return coords.x; }
public Point2D.Double getCoords() { return coords; }
public int getAxis() { return axis; }
public List<RailwayStation> getStations() { return
stations; }
public KDnode getLeft() { return left; }
public KDnode getRight() { return right; }
```

All getters/setters are constant-time operations

- Getter calls: O(1).
- Total: O(1).

## 2- KDtree class:

2.1 buildTreeFromStationList(List<RailwayStation>):~

Builds a KD-tree directly from a provided list of stations (used mainly for testing without the repository).

```
public void buildTreeFromStationList(List<RailwayStation>
stations) {
    if (stations == null || stations.isEmpty()) {
        this.root = null;
        return;
    }
    List<RailwayStation> byLat = new
ArrayList<RailwayStation>(stations);

byLat.sort(Comparator.comparingDouble(RailwayStation::getLa
titude));
    List<RailwayStation> byLon = new
ArrayList<RailwayStation>(stations);
byLon.sort(Comparator.comparingDouble(RailwayStation::getLo
ngitude));
    this.root = buildRec(byLat, byLon, 0);
}
```

- Copy station list → O(n)
- Sort by latitude → O(n log n)
- Sort by longitude → O(n log n)
- Build tree using recursive construction (next section)
- Total: O(n log n)

2.2 Buildtree normal method:

This method constructs the KD-tree using all stations stored in the StationRepo The analysis considers:

- n — total number of stations
- l — number of distinct latitudes/longitudes
- m — number of KD-tree nodes (m ≤ l)

```
public void buildTree() {
    StationRepository stationRepo =
StationRepository.getInstance();
    List<RailwayStation> sortedByLat = new ArrayList<>();
    for (RailwayStationLatitude n :
stationRepo.getLatitudeTree().inOrder()) {
        sortedByLat.addAll(n.getStations());
    }
    if (sortedByLat.isEmpty()) {
        root = null;
        return;
    }
    List<RailwayStation> sortedByLon = new ArrayList<>();
    for (RailwayStationLongitude n :
stationRepo.getLongitudeTree().inOrder()) {
        sortedByLon.addAll(n.getStations());
    }
    root = buildRec(sortedByLat, sortedByLon, 0);
}
```

- Access repository instance → O(1)
- Traverse latitude AVL tree (in-order) → O(l)
- Copy all stations from latitude buckets → O(n)
- Check for empty list → O(1)
- Traverse longitude AVL tree (in-order) → O(l)
- Copy all stations from longitude buckets → O(n)
- Build KD-tree using recursive construction (see Section 2.3) → O(n log n)

Since $l \leq n$, the tree construction step dominates the overall running time. Sorting is not performed in this function because the lists from the AVL trees are already ordered.

Total: O(n log n)

## 2.3 Recursive KD-tree construction (buildRec):

Recursively constructs the KD-tree by choosing medians, grouping identical coordinates, and dividing stations into left/right subtrees.

```
private KDnode buildRec(List<RailwayStation> byLat,
                        List<RailwayStation> byLon,
                        int depth) {
    if (byLat.isEmpty()) return null;
    int axis = depth % 2;
    List<RailwayStation> primary = (axis == 0 ? byLat :
byLon);
    int medianIndex = primary.size() / 2;
    RailwayStation median = primary.get(medianIndex);
    double mLat = median.getLatitude();
    double mLon = median.getLongitude();
    List<RailwayStation> sameCoords = new ArrayList<>();
    for (RailwayStation s : primary) {
        if (sameCoordinates(s, mLat, mLon))
sameCoords.add(s);
    }
    List<RailwayStation> leftByLat = new ArrayList<>();
    List<RailwayStation> rightByLat = new ArrayList<>();
    for (RailwayStation s : byLat) {
        if (sameCoordinates(s, mLat, mLon)) continue;
        if (isLeftOf(s, mLat, mLon, axis))
leftByLat.add(s);
        else rightByLat.add(s);
    }
    List<RailwayStation> leftByLon = new ArrayList<>();
    List<RailwayStation> rightByLon = new ArrayList<>();
    for (RailwayStation s : byLon) {
        if (sameCoordinates(s, mLat, mLon)) continue;
        if (isLeftOf(s, mLat, mLon, axis))
leftByLon.add(s);
        else rightByLon.add(s);
    }
    KDnode node = new KDnode(mLat, mLon, sameCoords, axis);
    node.setLeft(buildRec(leftByLat, leftByLon, depth +
1));
    node.setRight(buildRec(rightByLat, rightByLon, depth +
1));
    return node;
}
```

- Base case check: O(1).
- Median access: O(1).

- Loop for sameCoords: O(n).
- Loop for partitioning byLat: O(n).
- Loop for partitioning byLon: O(n).

The explanation:

It travels through all the stations to make the comparisons.

- KDnode creation: O(s log s).

Recurrence:

During the construction of the KD-tree, the buildRec method splits the list of stations into two parts every time it creates a new node. After selecting the median element, the algorithm divides the remaining stations into:

- one list for the left subtree (about n/2 elements)
- one list for the right subtree (about n/2 elements)

The algorithm must then build both subtrees. Each subtree has roughly half of the elements of the original list.

This is why the recurrence includes:

- T(n/2) for building the left subtree
- T(n/2) for building the right subtree

Before the recursive calls happen, the method also performs work proportional to the number of elements (scanning lists, separating elements, grouping equal-coordinate stations).
This work is linear in the input size → O(n).

For that reason, the recurrence relation becomes:

$$T(n) = 2T(n/2) + O(n)$$

At each recursive call, the algorithm divides the set of stations into two smaller subsets, each with approximately $n/2$ elements.

The number of times we can divide $n$by 2 is:

log2(n)

These operations are all linear in the number of elements processed, producing a cost of:

O(n)

So:

Total: T(n)= O(n) × O(log n) = O(n log n)

**2.4-** sameCoordinates(...)

Checks whether a station has exactly the same latitude and longitude as given coordinates

```
private boolean sameCoordinates(RailwayStation s, double
lat, double lon) {
    return Double.compare(s.getLatitude(), lat) == 0 &&
            Double.compare(s.getLongitude(), lon) == 0;
}
```

Only constant-time comparisons.

- Double comparisons: O(1).
- Total: O(1).

**2.5-** isLeftOf(...)

Determines whether a station should go into the left or right subtree based on the current splitting axis.

```
private boolean isLeftOf(RailwayStation s, double mLat,
double mLon, int axis) {
    if (axis == 0) {
        int cmpLat = Double.compare(s.getLatitude(), mLat);
        if (cmpLat < 0) return true;
        if (cmpLat > 0) return false;
        return Double.compare(s.getLongitude(), mLon) < 0;
    } else {
        int cmpLon = Double.compare(s.getLongitude(),
mLon);
        if (cmpLon < 0) return true;
        if (cmpLon > 0) return false;
        return Double.compare(s.getLatitude(), mLat) < 0;
    }
}
```

Only constant-time comparisons.

- Double comparisons: O(1).
- Total: O(1).

**2.6-** size()

Returns the number of KD-nodes in the tree.

```
public int size(){
    return sizeRec(root);
}

private int sizeRec(KDnode node){
    if (node == null) return 0;
    return 1 + sizeRec(node.getLeft()) +
```

```
sizeRec(node.getRight());
}
```

- Visits every node in the tree: O(m).
- Total: O(m).

**2.7-**height ()

Returns the height of the KD-tree.

```
public int height(){
   return heightRec(root);
}

private int heightRec(KDnode node){
   if (node == null) return 0;
   return 1 + Math.max(heightRec(node.getLeft()), heightRec(node.getRight()));
}
```

- Visits every node in the tree: O(m).
- Total: O(m).

**2.8** - bucketSizes()

Returns a list with the number of stations stored in each KD-node.

```
public List<Integer> bucketSizes() {
    List<Integer> sizes = new ArrayList<>();
    collectBucketSizes(root, sizes);
    return sizes;
}

private void collectBucketSizes(KDnode node, List<Integer>
sizes) {
    if (node == null) return;
    sizes.add(node.getStations().size());
    collectBucketSizes(node.getLeft(), sizes);
    collectBucketSizes(node.getRight(), sizes);
}
```

- Visits every node: O(m).
- sizes.add(): O(1) per node.
- Total: O(m).

**2.9-** distinctBucketSizes

Returns the unique station-count values found across all KD-nodes.

```
public Set<Integer> distinctBucketSizes() {
    return new HashSet<>(bucketSizes());
}
```

- bucketSizes() call: O(m).

- reate a new HashSet from the list of bucket sizes
  • iterate through all m elements → O(m)
  • insert each element into HashSet (average O(1)) → O(m)
- Total: O(m)

### **2.10-** inspectTree()

Prints basic structural statistics about the KD-tree for debugging purposes.

```
public void inspectTree() {
    System.out.println("KD-tree size  " + size());
    System.out.println("KD-tree height  " + height());
    System.out.println("Distinct bucket sizes " +
distinctBucketSizes());
}
```

- **Call size() → traverses all *m* KD-nodes → O(m)**
- **Call height() → traverses all *m* KD-nodes → O(m)**
- **Call distinctBucketSizes() → O(m) (HashSet version)**

Summary Table

| **Method** | **Complexity** |
|---|---|
| KDnode constructor | O(s log s) |
| KDnode getters/setters | O(1) |
| buildTree() | O(n log n) |
| buildRec() | O(n log n) |
| sameCoordinates() | O(1) |
| isLeftOf() | O(1) |
| size() | O(m) |
| height() | O(m) |
| bucketSizes() | O(m) |
| distinctBucketSizes() | O(m) |
| inspectTree() | O(m log m) |